

```
Terminal Help climate_app
! pubspec.yaml U weather.dart U x weather_provider.dart U weather_utils.dart U main.dart U home_screen.dart U
lib > models > weather.dart > Weather > Weather
1 class Weather {
2   final String city;
3   final double temp;
4   final String condition;
5   final String unit;
6
7   // Constructor para inicializar los datos
8   Weather({
9     required this.city,
10    required this.temp,
11    required this.condition,
12    required this.unit,
13  });
14 }
```

```
! pubspec.yaml U weather.dart U weather_provider.dart U x weather_utils.dart U main.dart U home_screen.dart U
lib > providers > weather_provider.dart > ...
1 import 'package:flutter_riverpod/legacy.dart';
2
3 import '../models/weather.dart';
4
5 class WeatherNotifier extends StateNotifier<Weather> {
6   WeatherNotifier() : super(Weather(
7     city: 'Querétaro',
8     temp: 24.0,
9     condition: 'Despejado',
10    unit: 'C',
11  ));
12
13   // Método para actualizar el estado por uno nuevo
14   void updateWeather(Weather newWeather) {
15     state = newWeather;
16   }
17 }
18
19 final weatherProvider = StateNotifierProvider<WeatherNotifier, Weather>((ref) {
20   return WeatherNotifier();
21 }); // StateNotifierProvider
```

```
! pubspec.yaml U weather.dart U weather_provider.dart U weather_utils.dart U X main.dart U home_screen.dart U
lib > utils > weather_utils.dart > ...
1 import 'package:flutter/material.dart';
2
3 String formatTemperature(double temp, String unit) {
4   // .toStringAsFixed(1) asegura que solo muestre un decimal
5   return '${temp.toStringAsFixed(1)}°$unit';
6 }
7
8 IconData getWeatherIcon(String condition) {
9   // Convertimos a minúsculas para evitar errores si viene como "Despejado" o "DESPEJADO"
10  switch (condition.toLowerCase()) {
11    case 'despejado':
12      case 'soleado':
13        return Icons.wb_sunny_rounded;
14    case 'lluvia':
15      case 'lluvioso':
16        return Icons.umbrella_rounded;
17    case 'nubes':
18      case 'nublado':
19        return Icons.cloud_rounded;
20    case 'tormenta':
21        return Icons.thunderstorm_rounded;
22    case 'nieve':
23        return Icons.ac_unit_rounded;
24    default:
25      // Un ícono neutral por si el estado no coincide con ninguno de los anteriores
26      return Icons.device_thermostat_rounded;
27  }
28 }
```

```
! pubspec.yaml U weather.dart U weather_provider.dart U weather_utils.d
lib > main.dart > main
1 import 'package:flutter/material.dart';
2 import 'package:flutter_riverpod/flutter_riverpod.dart';
3
4 import 'screens/home_screen.dart';
5
6 Run | Debug | Profile
7 void main() {
8   // Envolvemos la app con ProviderScope para activar Riverpod
9   runApp(
10     const ProviderScope(
11       child: MyApp(),
12     ), // ProviderScope
13   );
14
15   class MyApp extends StatelessWidget {
16     const MyApp({Key? key}) : super(key: key); // Parameter 'key' could be
17
18     @override
19     Widget build(BuildContext context) {
20       return MaterialApp(
21         title: 'Climate App',
22         debugShowCheckedModeBanner: false,
23         theme: ThemeData(
24           primarySwatch: Colors.blue,
25           useMaterial3: true,
26         ), // ThemeData
27         home: const HomeScreen(),
28       ); // MaterialApp
29     }
30   }
```




